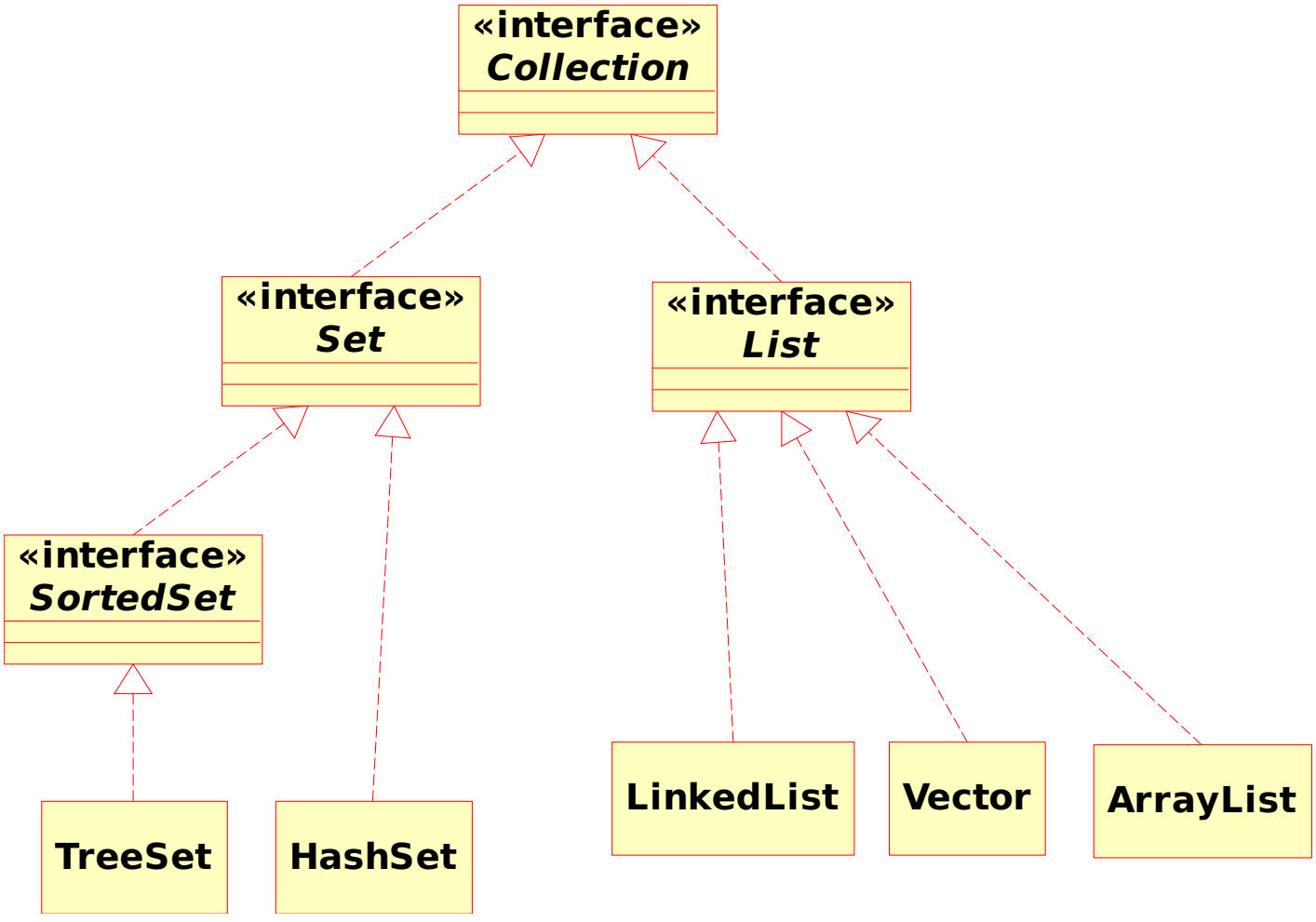
DD1385
Programutvecklingsteknik
Fler bilder till föreläsning 6

Innehåll

- Javas objektsamlingar
- Generiska typer
- Mönstret Iterator
- Natural order
- Mönstret Template Method
- Mönstret Composite
- Mönstret Factory-teknik

Collection

Javas bibliotheksklasser för objektsamlingar,
en del av klasshierarkin:



List

- Numrerad samling objekt
- Dynamisk
  (kan växa och krympa, jfr []-array)
- Insättning och borttagning görs var som helst i listan

Ett objekt kan finnas på flera ställen i en lista

Det gäller []-arrayer också !

ArrayList, Vector

- Lagras m.h.a. []-arrayer
- Bäst för snabb åtkomst/ändring av listelement.

LinkedList

- Lagras länkat
- Bäst för snabb insättning/borttagning av listelement

Generiska typer

En objektsamling kan skapas utan att elementtypen anges:

```
ArrayList lista = new ArrayList();
```

Element av typ `Object` antas men varningsmeddelande ges då listan används.

Collection-klasserna kan ges typparameter

```
ClassName<ElementType>
```

Generiska typer, forts.

Större säkerhet och enklare hantering med

```
CollType<ElementTyp> coll =
        new CollType<ElementTyp>();
```

Till exempel:

```
ArrayList<String> lista =
        new ArrayList<String>();
```

Endast `String`-objekt kan sättas in i listan

Några metoder i `Collection`.

Alla implementerande klasser har dessa

```
add(obj)
addAll(coll)
iterator()
removeAll(coll)
retainAll(coll)
toArray()
remove(obj)
isEmpty()
size()
```

Mönstret Iterator

- Ger elementen ur en komplicerad struktur
  i sekvens
- Avslöjar ej detaljer om lagringen
- Implementerar ett `Iterator` – gränssnitt, t.ex.

```
interface Iterator {
   Object first();
   Object next();
   boolean isDone();
   Object current();
}
```

Iteratorn känner till den underliggande strukturen
och ”vet” hur långt den kommit i iterationen

Iteratorn i Java-API:n

```
interface Iterator<T> {
  T next();
  boolean hasNext();
  void remove();
}
```

Alla objektsamlingar har Iterator !!

Några metoder i `List`.

add(obj)	sätt in sist
add(k,obj)	sätt in på plats k
get(k)	referens till objektet på plats k
	returtyp Object eller T
	T = typparametern till objektsamlingen
remove(k)	tag bort elementet på plats k

Set

- Ett objekt kan bara finnas med en gång.
- Använder `equals()` från `Object`

HashSet

- Snabb åtkomst
- Använder `hashCode()` från `Object`

SortedSet

- Hålls sorterad
- Elementen måste implementera interfacet `Comparable<T>` eller `Comparable`

Comparable utan typparameter

```
public interface Comparable {
    public int compareTo (Object obj);
}
```

Comparable med typparameter

```
public interface Comparable<T> {
    public int compareTo (T obj);
}
```

`compareTo()` returnerar <0, 0 eller >0 beroende på ordningen mellan elementen.

`t1.compareTo(t2)` <0 om `t1` kommer före `t2`.

Exempel med Set

Generera
`goal` st olika tal från intervallet `1..range`.

tries räknar antalet försök.

range $\geq$ goal nödvändigt.

Använd subklassen HashSet.

- Slumpa tal, lägg i en mängd
- När mängden har goal st tal är det klart
  (vi vet ju att det är olika tal)

```
HashSet<Integer> hset =
  new HashSet<Integer>();

int tries = 0;

while (hset.size() < goal){
  int next =
    (int)(Math.random()*range+1);
  hset.add(next);    // autoboxing
  tries++;
}
```

Skriv ut talen ur hset

Med for-sats:

```
for (int i : hset)
    System.out.print(" " + i);
System.out.println();
```

Med iterator:

```
Iterator<Integer> iter = hset.iterator();

while (iter.hasNext())
    System.out.print(" " + iter.next());
System.out.println();
```

Varning för osäker operation:

Att ta bort element ur en lista medan man loopar över listan.

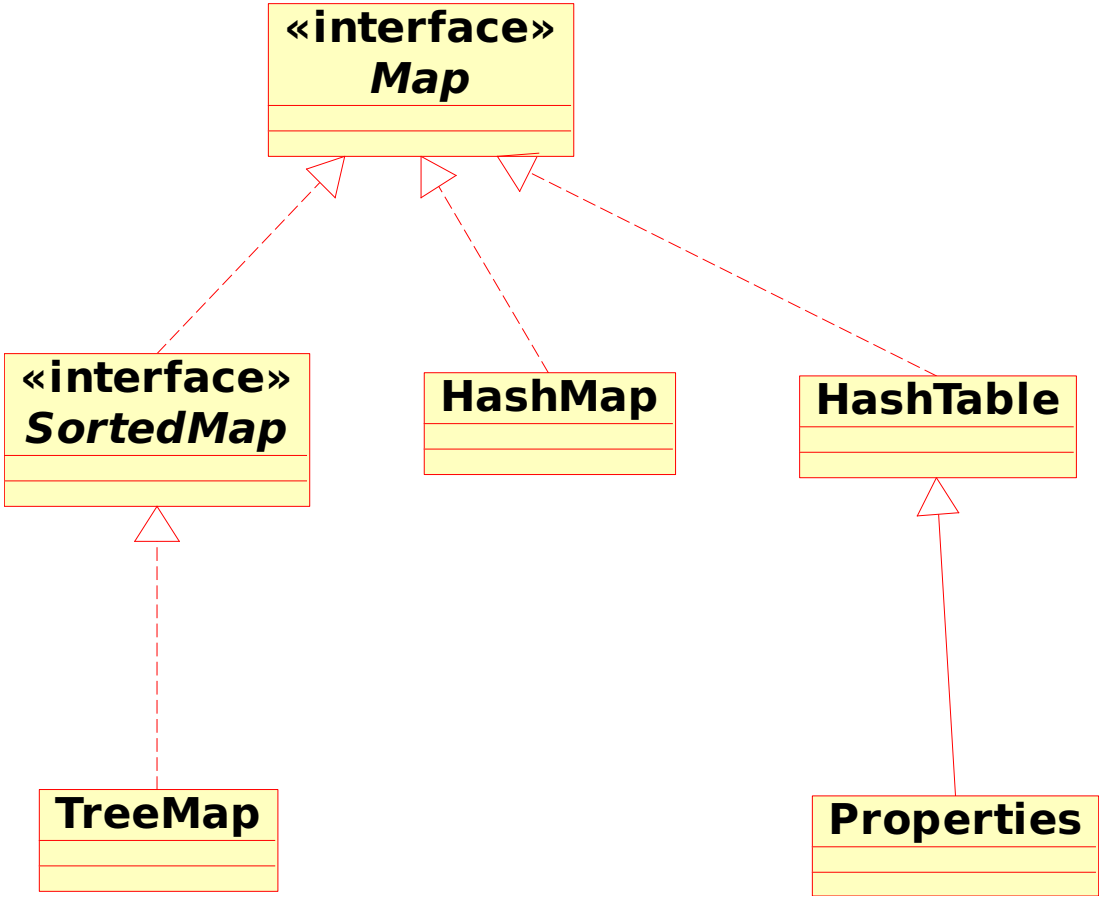
Via `iterator()` i Java-collections går det bra:

Tag bort alla 5-multipler ur hset via iter:

```
while (iter.hasNext()){
   int x = iter.next(); // autoboxing
   if (x%5 == 0)
      iter.remove();
}
```

Map

Bibliotheksklasser för avbildningstabeller,
en del av klasshierarkin



Map

Objekt lagras tillsammans med en nyckel som också är ett objekt.

Map liknar dictionaries i Python

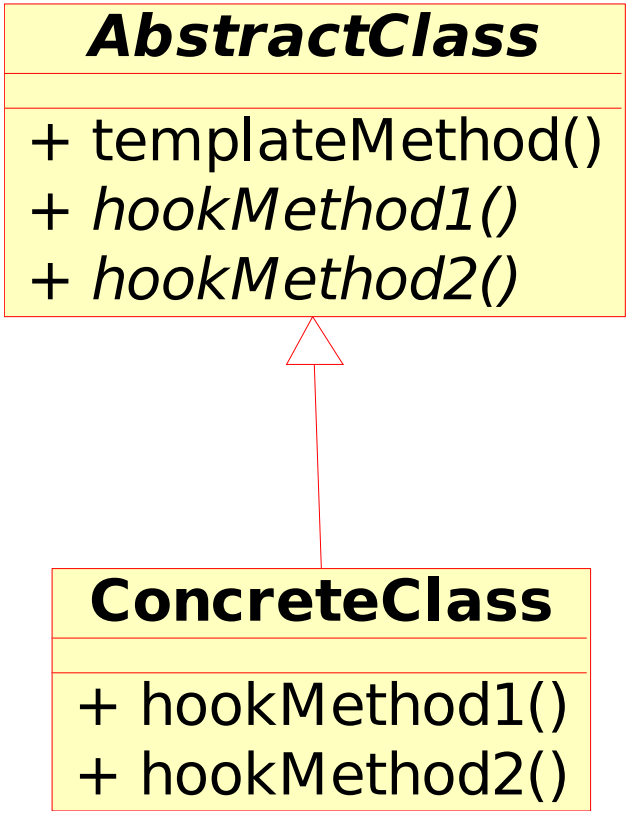
Några metoder i Map

`put(key,val)`	lägg in val med nyckel key
`remove(key)`	tag bort
`get(key)`	ger referens till objektet
`keySet()`	alla nycklar
`size()`	antal element

Designmönster: Template Method

Skjut upp delar av en algoritm till subklasser

Template Method

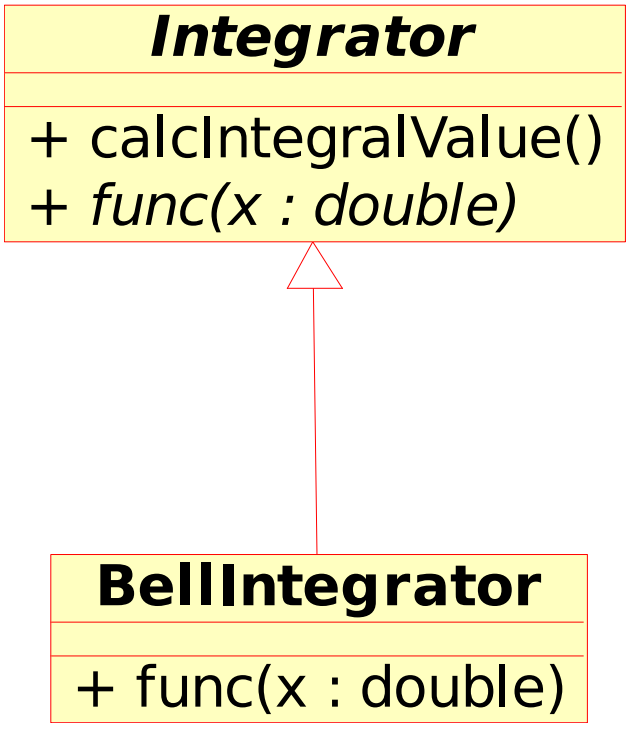


`templateMethod()`
beskriver en konkret algoritm

hookMethod1()
hookMethod2()
används i `templateMethod()`
men är abstrakta!

I `ConcreteClass`
definieras konkreta
`hookMethod1()`
`hookMethod2()`

Template Method – exempel



Integralen beräknas,
`func(x)` anropas.

men `func(x)` definieras
först här.

Template Method - exempel : Numerisk integrering

```
abstract class Integrator {
    :

  void set (double low, double high, double dx){...}

  double calcIntegralValue() {
        :
    for (int x=low; x<high; x = x+dx ){
          :
       sum += func(x);
    }
  }
  abstract double func(double x);
}
```

Vilken funktion integreras ?

Funktion att integrera bestäms i subklass

```
class BellIntegrator extends Integrator{
  double func(double x) {
    return Math.exp(-x*x);
  }
```

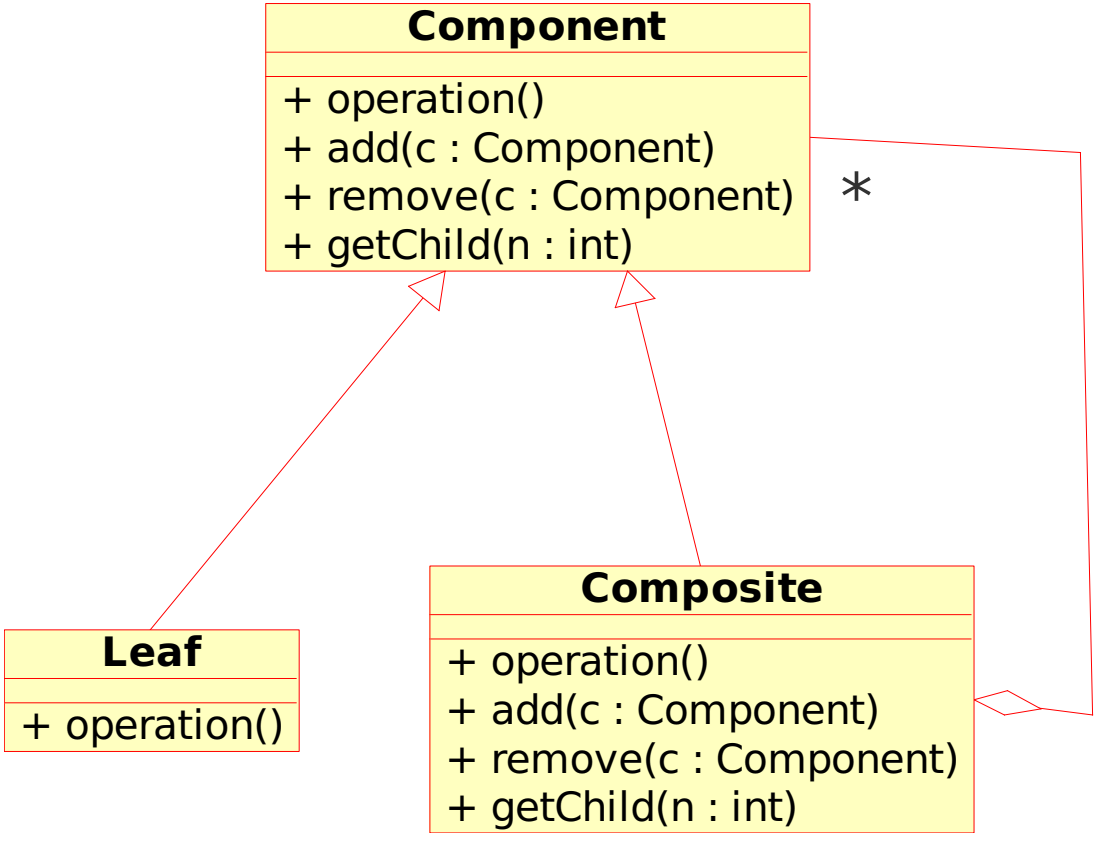
Testa i main-metod:

```
  public static void main(String[] a) {
    BellIntegrator bi = new BellIntegrator();
    bi.set(0.1, 0.5, 0.001);
    System.out.println(bi.integralValue());
    bi.set(0, 1, 0.001);
    System.out.println(bi.integralValue());
  }
}
```

Designmönster: Composite

- Objekt ordnas i en trädstruktur
- Operationer kan utföras på enskilda objekt eller grupper av objekt

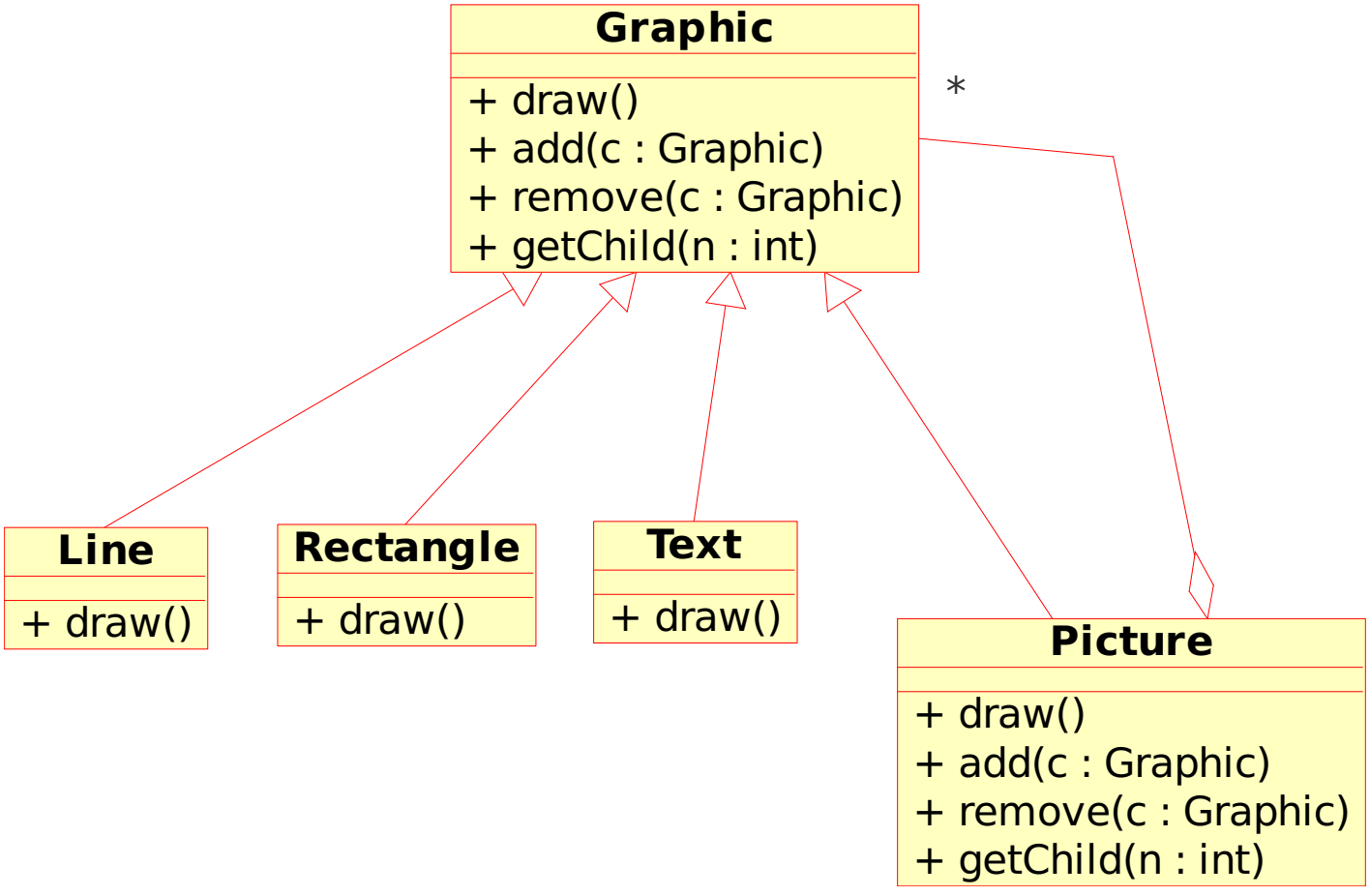
Composite



Composite
har en lista med Components

```
operation():
for (o:lista)
o.operation()
```

Composite-exempel



Factory-teknik

- Objekt skapas *inte* med
  `new A(...)`
- Objekt skapas med fabriksmetod, t.ex.
  `A.getInstance(...)`

Varför ?

Skäl för factory-teknik

- Säkerhet: - antalet objekt ska kontrolleras
- Säkerhet o/e bekvämlighet:
  - en särskild subklass ska användas
- Effektivitet
  - kanske inte nödvändigt att skapa nytt objekt
- Flexibilitet - t.ex. om flera konstruktorer med samma signatur önskas

```
new Point(x,y)
new Point(r,fi)
```

båda är Point(double, double)

```
class Point {
    private double x, y, r, fi;

    private Point (){}

    static Point createPolar
                (double r, double fi)
    {...}

    static Point createCartesian
                (double x, double y)
    {...}
    ......... // fler metoder
}
```

Fabriksmetoden createPolar

```
static Point createPolar(double r, double fi){
   Point p = new Point();
   p.r = r; p.fi = fi;
   p.x = r*Math.cos(fi);
   p.y = r*Math.sin(fi);
   return p;
}
```

Fabriksmetoden createCartesian

```
static Point createCartesian
                              (double x, double y){
  Point p = new Point();
  p.x = x; p.y = y;
  p.r = Math.sqrt(x*x + y*y);
  p.fi = Math.atan2(y,x);
  return p;
}
```

eller

```
static Point createCartesian
                              (double x, double y){
  Point p = new Point();
  p.setCartesian(x,y);
  return p;
}
```

Skapa nya Point-objekt

```
Point p1 = Point.createCartesian (4.3 ,5.8);
Point p2 = Point.createPolar (6.0 ,0.6);
```